Project Data Build

This notebook provides details on how the data for the project was curated within VUMC's dataset. The data is hosted in Azure databricks, and as such portions of the notebook involve code specific to this environment. While individual code cells may be useful for reference, the SQL script used for the data curation and transformation will be marked with **[SQL SCRIPT]**.

Assumptions and Definitions

Data Model

The data source at VUMC is uses the OMOP common data model. As such the names of tables may differ, depending on your local data source.

Data Source and Working Data Names

Within the SQL scripts, there are references to a data source (`source_server`.`source_database`) and a working database (`working_server`.`working_database`). In your environment, `source_server`.`source_database`, will correlate to the source database from which structured data will be pulled for use in your local database, `working_server`.`working_database`. In practice, `source_server` and `working_server` may actually be the same database server, but they are separated there to avoid ambiguity.

Notes about SQL Conventions

ANSI-SQL

The database queries are written using ANSI-standard SQL. If you use Microsoft SQL Server, the major difference will be in how the fully qualified table name is referenced. In this case, a reference to

```
select * from `source_server`.`source_database`.`source_table` limit 10;
```

is equivalent to

```
select top 10 * from [source_server].[source_database].[source_table]
```

Databricks-specific Conventions

There are some script conventions used solely to optimize Databricks queries. Fortunately, optimizations such as `/*+ BROADCAST(note) */` will be ignored as comments by other environments.

Table / Views Definitions

View: Concepts

In the OMOP CDM, the central table that defines concepts across different vocabularies is the `Concepts` table. If you are not using an OMOP-based data source, you will not have this table.

[SQL SCRIPT]

OMOP Concept Table

```
SELECT concept_id,
       concept_name,
       domain_id,
       vocabulary_id,
       concept_class_id
FROM `source_server`.`source_database`.`concept`
WHERE concept_id = ...
```

Table: Patient Cohort

The cohort table should provide the list of unique identifiers for patients in the host system. For purposes of this code `person_id` is the unique identifier in the system for a patient, and it used to find related notes.

[SQL SCRIPT]

Cohort Table Query

```
SELECT deid_pat_id,
       mrn,
       person_id
FROM `working_server`.`working_database`.`cohort`
```

View: Patient Notes over Entire Project Date Range

View joining notes to patient over the upper and lower date for the entire project.

[SQL SCRIPT]

View creation query

```
CREATE OR REPLACE VIEW vw_cohort_note AS
SELECT /*+ BROADCAST(note) */ note.note_id,
                              note.person_id,
                              note.note_type_concept_id,
                              note.provider_id,
                              note.visit_occurrence_id,
                              note.note_date,
                              note.note_title,
                              note.note_source_value,
                              note.x_source,
                              note.x_part_num,
                              note.x_uniq
FROM `working_server`.`working_database`.`cohort` AS cohort
INNER JOIN
  (SELECT n.note_id,
          n.person_id,
          n.note_type_concept_id,
          n.provider_id,
          n.note_title,
          n.note_date,
          n.visit_occurrence_id,
          n.note_source_value,
          n.x_source,
          n.x_part_num,
          n.x_uniq
   FROM `source_server`.`source_database`.`note` AS n
   WHERE n.note_date BETWEEN '2010-01-01' AND '2024-12-31') AS note
    ON cohort.person_id = note.person_id
```

View: Visit Occurrences

In order to determine whether or not the note date falls between the upper/lower bound of the visit, join to the visit table to get the visit date for comparison.

[SQL SCRIPT]

View creation query

```
CREATE OR REPLACE VIEW vw_cohort_visit_occurrence AS
SELECT /*+ BROADCAST(notes) */ DISTINCT visit_occur.visit_occurrence_id,
                                        visit_occur.visit_concept_id,
                                        visit_occur.person_id,
                                        visit_occur.discharge_to_concept_id,
                                        visit_occur.provider_id,
                                        visit_occur.visit_start_date,
                                        visit_occur.visit_end_date
FROM `working_server`.`working_database`.`vw_cohort_note` AS notes
INNER JOIN `source_server`.`source_database`.`visit_occurrence` AS visit_occur
    ON notes.visit_occurrence_id = visit_occur.visit_occurrence_id
AND notes.visit_occurrence_id IS NOT NULL
```

View: Providers

The provider who created the note.

[SQL SCRIPT]

View creation query

```
CREATE OR REPLACE VIEW vw_cohort_provider AS
SELECT /*+ BROADCAST(provider) */ DISTINCT provider.provider_id,
                                           provider.specialty_concept_id
FROM `working_server`.`working_database`.`vw_cohort_note` AS note
INNER JOIN `source_server`.`source_database`.`provider` AS provider
    ON note.provider_id = provider.provider_id
WHERE note.provider_id IS NOT NULL
```

Note Groups

Note groups as defined in the "NLP Extraction Instruction" document are at minumum defined by a set of one or more note titles. However, there may be additional features defining notes within each group, which require additional logic.

The following view exposes features which were used in later note group creation.

[SQL SCRIPT]

View creation query

```
CREATE OR REPLACE VIEW vw_cohort_corpus_features AS
SELECT notes.note_id,
       notes.person_id,
       notes.note_type_concept_id,
       notes.note_title,
       notes.note_source_value,
       notes.x_source,
       visit_occurrence.visit_concept_id,
       concept.concept_name AS discharge_to_concept_name,
       provider.specialty_concept_id AS associated_provider_specialty_id
FROM vw_cohort_note AS notes
INNER JOIN `working_server`.`working_database`.`cohort` AS cohort
    ON notes.person_id = cohort.person_id
LEFT JOIN
  (SELECT visit_occurrence_id,
          visit_concept_id,
          discharge_to_concept_id,
          provider_id
   FROM vw_cohort_visit_occurrence
   WHERE discharge_to_concept_id IS NOT NULL
     OR provider_id IS NOT NULL) AS visit_occurrence
    ON notes.visit_occurrence_id = visit_occurrence.visit_occurrence_id
LEFT JOIN
  (SELECT concept_id,
          concept_name
   FROM `source_server`.`source_database`.`concept`) AS concept
    ON visit_occurrence.discharge_to_concept_id = concept.concept_id
LEFT JOIN
  (SELECT provider_id,
          specialty_concept_id
   FROM vw_cohort_provider) AS provider
    ON visit_occurrence.provider_id = provider.provider_id
```

Creating Note Groups

Based upon data in `vw_cohort_corpus_features`, the following note groups will are defined:

- `AdmissionNotesTable` - the group of notes associated with *Admissions Notes*
- `CommunicationEncounterNotesTable` - the group of notes associated with *Patient Communications Notes*

- `DischargeSummaryNotesTable` - the group of notes associated with *Discharge Summaries Notes*
- `ECGImpressionNotesTable` - the group of notes associated with *ECG Impressions Notes*
- `EmergencyDepartmentNotesTable` - the group of notes associated with *Emergency Department Notes*
- `InpatientNotesTable` - the group of notes associated with *Inpatient Notes*
- `OutpatientNotesTable` - the group of notes associated with *Outpatient Notes*
- `ProblemListNotesTable` - the group of notes associated with *Problem List Notes*

Admission Note Group

[SQL SCRIPT]

admission notes

```
CREATE OR REPLACE VIEW vw_cohort_admission_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((note_type_concept_id IN (44814638))
      OR (instr(lower(note_source_value), 'admission') > 0)
      OR (instr(lower(note_title), 'admission') > 0))
```

Communication Encounter Note Group

[SQL SCRIPT]

communication encounter notes

```
CREATE OR REPLACE VIEW vw_cohort_communication_encounter_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((instr(lower(note_title), 'communication') > 0))
```

Discharge Summary Note Group

[SQL SCRIPT]

discharge summary notes

```
CREATE OR REPLACE VIEW vw_cohort_discharge_summary_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((lower(note_source_value) IN ('discharge summary'))
      OR (instr(lower(note_title), 'discharge summary') > 0))
```

ECG Impression Note Group

[SQL SCRIPT]

ecg impression notes

```
CREATE OR REPLACE VIEW vw_cohort_ecg_impression_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((lower(note_title) LIKE '%ecg%impression%'))
```

Emergency Department Note Group

[SQL SCRIPT]

emergency department notes

```
CREATE OR REPLACE VIEW vw_cohort_emergency_department_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((associated_provider_specialty_id IN (38003849,
                                             38003695,
                                             38003728,
                                             38003850,
                                             45756808,
                                             38003847,
                                             38004510,
                                             38003846))
       OR (note_type_concept_id IN (44814646))
       OR (visit_concept_id IN (9203))
       OR (instr(lower(note_source_value), 'emergency') > 0)
       OR (instr(lower(note_title), 'emergency') > 0))
```

Inpatient Note Group

[SQL SCRIPT]

inpatient notes

```
CREATE OR REPLACE VIEW vw_cohort_inpatient_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((associated_provider_specialty_id IN (8970,
                                             8717))
       OR (note_type_concept_id IN (44814639))
       OR (visit_concept_id IN (9201))
       OR (instr(lower(discharge_to_concept_name), 'inpatient') > 0)
       OR (instr(lower(note_title), 'inpatient') > 0))
```

Outpatient Note Group

[SQL SCRIPT]

outpatient notes

```
CREATE OR REPLACE VIEW vw_cohort_outpatient_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((instr(lower(note_title), 'outpatient') > 0)
       OR (note_type_concept_id IN (44814640))
       OR (visit_concept_id IN (9202)))
```

Problem List Note Group

[SQL SCRIPT]

problem list notes

```
CREATE OR REPLACE VIEW vw_cohort_problem_list_notes_group AS
SELECT DISTINCT note_id
FROM vw_cohort_corpus_features
WHERE ((note_type_concept_id IN (2002404173,
                                 2002603124)))
```

NLP Pipeline

The local NLP pipeline was implemented in Python using MedspaCy, where logic was constructed to search for terms (see "NLP Extraction Instruction") based upon note group membership.

Support Tables

Table: note_group_map

To facilitate this, it is useful to create a table (or local dataset) mapping a unique ID to each note group type.

```
CREATE OR REPLACE TABLE note_group_map (
  note_group_id INT,
  note_group STRING
);

INSERT INTO note_group_map (note_group_id, note_group) VALUES
  (1, 'admission_notes_group'),
  (2, 'communication_encounter_notes_group'),
  (3, 'discharge_summary_notes_group'),
  (4, 'ecg_impression_notes_group'),
  (5, 'emergency_department_notes_group'),
  (6, 'inpatient_notes_group'),
  (7, 'outpatient_notes_group'),
  (8, 'problem_list_notes_group');
```

Table: notes_to_process

Next create logic that both maps each unique note id to its note group membership set as well as filters the notes based on the date range defined for each term.

```
CREATE OR REPLACE TABLE notes_to_process AS
SELECT note_id, note_group_id
FROM (
  SELECT note_id, CAST(1 AS TINYINT) as note_group_id
  FROM vw_cohort_admission_notes_group
  UNION ALL
  SELECT note_id, CAST(2 AS TINYINT) as note_group_id
  FROM vw_cohort_communication_encounter_notes_group
  UNION ALL
  SELECT note_id, CAST(3 AS TINYINT) as note_group_id
  FROM vw_cohort_discharge_summary_notes_group
  UNION ALL
  SELECT note_id, CAST(4 AS TINYINT) as note_group_id
  FROM vw_cohort_ecg_impression_notes_group
  UNION ALL
  SELECT note_id, CAST(5 AS TINYINT) as note_group_id
  FROM vw_cohort_emergency_department_notes_group
  UNION ALL
  SELECT note_id, CAST(6 AS TINYINT) as note_group_id
  FROM vw_cohort_inpatient_notes_group
  UNION ALL
  SELECT note_id, CAST(7 AS TINYINT) as note_group_id
  FROM vw_cohort_outpatient_notes_group
  UNION ALL
  SELECT note_id, CAST(8 AS TINYINT) as note_group_id
  FROM vw_cohort_problem_list_notes_group
```

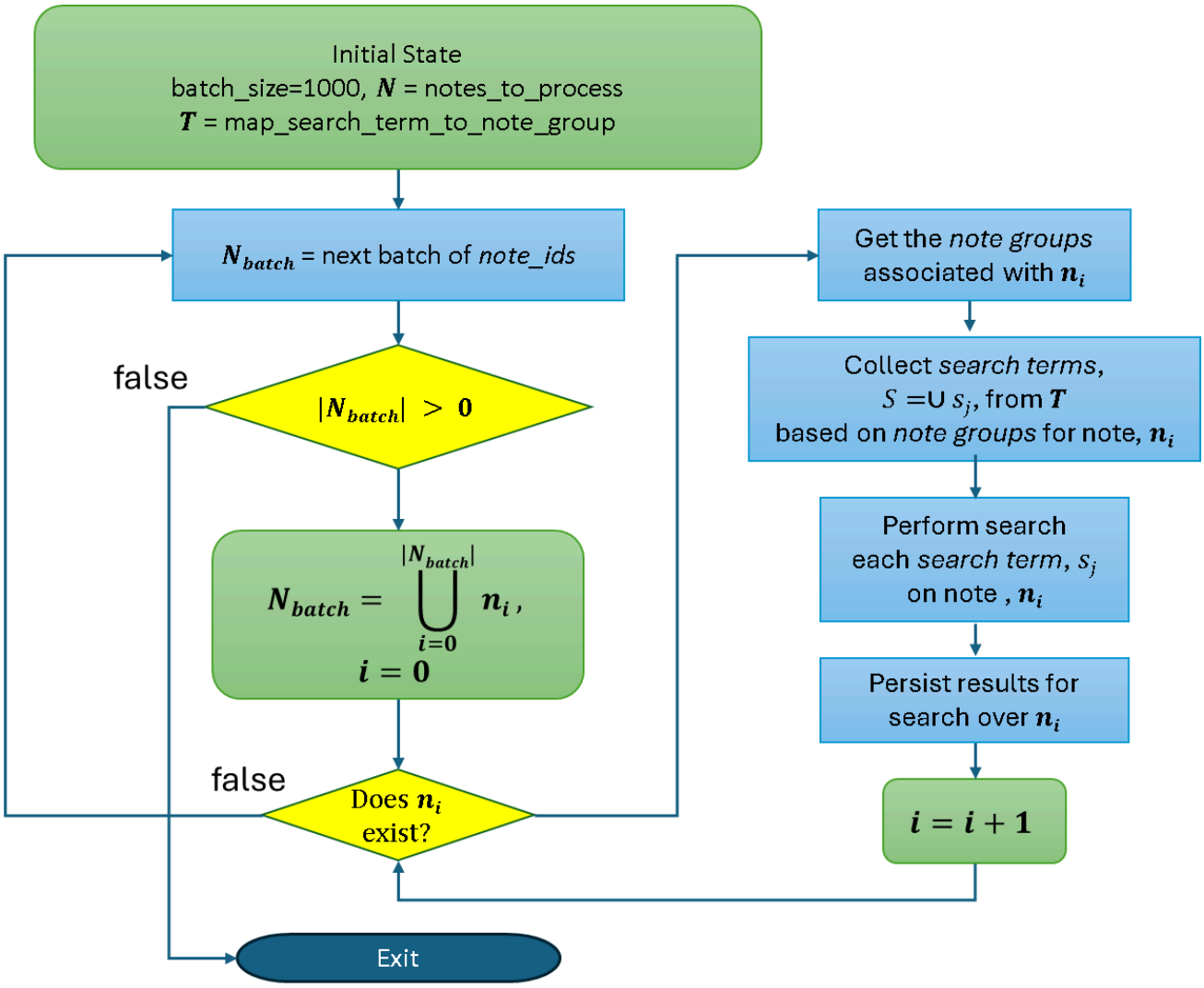
```
)
GROUP BY note_id, note_group_id;
```

Table: map_search_term_to_note_group

Finally, create a table that maps terms to note groups, `map_search_term_to_note_group` (you can use `map_search_term_to_note_group.csv` to build this).

search_term_to_note_groups_id	search_term	admission_notes_group	communication_encounter_notes_group	discharge_summary_notes_group	ecg_impression_notes_group	emergency_department_notes_group	inpatient_notes_group	outpatient_notes_group	problem_list_notes_group
1	heart failure with reduced ejection fraction	1	0	1	1	1	1	1	1
2	hfref	1	0	1	1	1	1	1	1
3	heart failure with preserved ejection fraction	1	0	1	1	1	1	1	1
4	hfpef	1	0	1	1	1	1	1	1
5	hyponatremia	1	1	1	0	1	1	1	1
6	low sodium	1	1	1	0	1	1	1	1
7	low na	1	1	1	0	1	1	1	1
...									

Workflow


Initial State
batch_size=1000, N = notes_to_process
T = map_search_term_to_note_group
N_{batch} = next batch of *note_ids*
false
$|N_{batch}| > 0$
$N_{batch} = \bigcup_{i=0}^{|N_{batch}|} n_i$,
$i = 0$
false
Does n_i exist?
Get the *note groups* associated with n_i
Collect *search terms*, $S = \cup s_j$, from T based on *note groups* for note, n_i
Perform search each *search term*, s_j on note , n_i
Persist results for search over n_i
$i = i + 1$
Exit